DocAI — Intelligent Document Extraction Pipeline

A production-ready FastAPI backend that ingests documents (PDF, images, text), performs OCR + LLM-based extraction, classifies document type, verifies outputs against source text, persists results to MongoDB, and exposes a lightweight review dashboard + feedback loop that can be exported as JSONL for model tuning.

> **Heads-up on credentials:** The repo currently contains `.env` and some hard-coded secrets in `Backend/test_openai.py` and `Backend/Dockerfile` (ENV `MONGODB_URI`). **Rotate these immediately** and remove them from source control. See **Security & Secrets** below for best practices.

Table of Contents

- Features
- Architecture
- Repository Layout
- Prerequisites
- Quick Start (Local)
- Environment Variables
- Running with Docker
- API Reference
- Data Model (MongoDB)
- Dashboard & Human-in-the-Loop
- Training Data Export (JSONL)
- Logging & Debugging
- Testing
- Security & Secrets
- Performance Notes
- Troubleshooting
- Roadmap
- License

Features

- **OCR first** with fallbacks: `pdf2image` + Tesseract / TrOCR; simple text decode fallback for text files. (`Backend/ocr_utils.py`)
- **Document classification** via OpenAI Chat Completions (configurable). (`Backend/document_classifier.py`)
- **Hybrid extraction**:
- Heuristic + OCR text cleaning (`clean_text`) and strategy routing (`Backend/strategy_router.py`).
- Optional **Donut** (DocVQA) model for visual JSON extraction. (`Backend/donut_extractor.py`)

- **Merge engine** to combine primary + fallback outputs without clobbering confident fields. (`Backend/merge_engine.py`)
- **Verification**: aligns extracted fields back to OCR text and computes a confidence score / unmatched fields. (`Backend/verifier.py`)
- **Persistence**: MongoDB (`docai`) with collections for OCR jobs and feedback. (`Backend/database.py`, `main.py`)
- **FastAPI** endpoints for upload, feedback, and review dashboard (`/dashboard`). (`Backend/main.py`, `Backend/templates/dashboard.html`)
- **Feedback export** to JSONL for iterative tuning. (`Backend/export_feedback.py`)

Architecture

```
[Client]
   |  upload PDF/Image/Text
   v
[FastAPI `main.py`]  -- receives file -->  [ocr_utils.extract_text_from_file]
   |                                            |  OCR/Tesseract/TrOCR
   |                                            v
   |                                raw_text + meta (engine, pages)
   |--> [strategy_router.extract_text_and_classify]
   |        |--> [document_classifier.classify_document]  (OpenAI)
   |        |--> layout hints / route strategies
   |--> [donut_extractor.extract_donut_data]  (optional visual JSON)
   |--> [merge_engine.merge_outputs]          (hybrid JSON)
   |--> [verifier.verify_extraction_alignment] + confidence
   v
[database.save_analysis]  --> MongoDB (docai)
   |
   +--> [/dashboard] human review + feedback --> [/feedback]
                                     |
                                     +--> export_feedback.py --> JSONL for
training
```

Visual Architecture (Mermaid)

> These render in GitHub/GitLab/VS Code with Mermaid enabled. You can also export PNGs/SVGs via the Mermaid CLI.

High-Level Data Flow

```
flowchart TD
    A[User / Client] -->|Upload file| B[FastAPI main.py]
    B --> C[ocr_utils
OCR + text extraction]
    C --> D[strategy_router
layout hints]
```

```
    D --> E[document_classifier
(OpenAI)]
    D --> F[donut_extractor
DocVQA]
    D --> G[LLM/Heuristic
text extractor]
    E --> H[merge_engine]
    F --> H
    G --> H
    H --> I[verifier
confidence + alignment]
    I --> J[(MongoDB docai)]
    J --> K[Dashboard /feedback]
    K --> L[[export_feedback.py
JSONL for training]]
```

Upload → Extraction → Review (Sequence)

```
sequenceDiagram
    participant U as User
    participant API as FastAPI
    participant OCR as OCR
    participant CLF as Classifier
    participant EXT as Extractors
    participant VER as Verifier
    participant DB as MongoDB
    participant UI as Dashboard

    U->>API: POST /upload (file)
    API->>OCR: extract_text_from_file
    OCR-->>API: raw_text, meta
    API->>CLF: classify_document(raw_text)
    CLF-->>API: document_type
    API->>EXT: run strategies (Donut / LLM)
    EXT-->>API: candidate JSONs
    API->>VER: verify + score
    VER-->>API: verification_score, aligned JSON
    API->>DB: save_analysis
    API-->>U: job_id, summary
    U->>UI: GET /dashboard
    UI->>API: POST /feedback
    API->>DB: store feedback
    U->>API: export JSONL
```

Data Model (ERD)

```
erDiagram
    OCR_RESULTS ||--o{ EXTRACTION_FEEDBACK : has
    OCR_RESULTS {
```

```
        string job_id PK
        string filename
        datetime timestamp
        string ocr_source
        string document_type
        float verification_score
        json final_structured_data
        json meta
    }
    EXTRACTION_FEEDBACK {
        string job_id FK
        string field_path
        string correct_value
        string note
        datetime timestamp
    }
```

Deployment Topology

```
flowchart LR
    subgraph Client Side
      U[Reviewer / Uploader]
    end
    subgraph Server Side
      API[FastAPI + Uvicorn]
      DONUT[(Donut weights)]
      TES[Tesseract/Poppler]
      LOGS[(Structured Logs)]
    end
    DB[(MongoDB Atlas)]

    U --> API
    API --> TES
    API --> DONUT
    API --> LOGS
    API --> DB
```

Export visuals as files (optional):

```
npm i -g @mermaid-js/mermaid-cli
mmdc -i docs/flow.mmd -o docs/flow.png   # create .mmd files from the
snippets above
```

Repository Layout

```
Backend/
  ├─ main.py                    # FastAPI app: upload, feedback, dashboard
  ├─ ocr_utils.py               # OCR / text extraction + fallbacks
  ├─ strategy_router.py         # Chooses OCR/LLM path, basic layout
heuristics
  ├─ document_classifier.py     # OpenAI-based doc type classifier
  ├─ donut_extractor.py         # Donut DocVQA pipeline (optional)
  ├─ merge_engine.py            # Deep merge of multiple extraction outputs
  ├─ verifier.py                # Field-to-source alignment + confidence
  ├─ database.py                # Mongo connection + save helper
  ├─ export_feedback.py         # Builds verified JSONL from feedback
  ├─ templates/
  │   └─ dashboard.html         # Bootstrap review UI
  ├─ requirements.txt
  ├─ Dockerfile
  ├─ test_mongo.py              # Quick connectivity check
  └─ test_openai.py             # **Remove secrets; use env var**
Frontend/                       # (placeholder for future UI)
Readme.txt                      # (empty placeholder)
```

Prerequisites

- **Python** 3.10+ (tested with 3.10/3.11)
- **Tesseract OCR**, **poppler**/image libs (installed automatically in Docker; for local install see Troubleshooting)
- **MongoDB** (Atlas or self-hosted)
- **OpenAI** API key

Quick Start (Local)

1. **Clone & enter project**

   ```
   cd docai_project/Backend
   ```

2. **Create and activate a virtualenv**

   ```
   python -m venv .venv
   source .venv/bin/activate   # Windows: .venv\Scripts\activate
   ```

3. **Install dependencies**

```
pip install --upgrade pip
pip install -r requirements.txt
```

4. **Set environment variables** (see next section) — do **not** commit `.env`.
5. **Run the API**

```
uvicorn main:app --reload --port 8000
```

6. **Open the dashboard**
7. Visit: `http://localhost:8000/dashboard`

Environment Variables

Create a `.env` (not checked in) in `Backend/` with:

```
OPENAI_API_KEY=sk-...              # Your OpenAI key (no quotes)
MONGODB_URI=mongodb+srv://...      # Connection string to your DB
```

> **Remove** any hard-coded keys from `test_openai.py` and `Dockerfile`. Prefer `docker run -e VAR=...` or `.env` files.

Running with Docker

> The current `Backend/Dockerfile` bakes a sample `MONGODB_URI` into the image. Replace that with an environment variable at run time.

Build:

```
docker build -t docai-backend ./Backend
```

Run:

```
docker run --rm -p 8000:8000
  -e OPENAI_API_KEY=$OPENAI_API_KEY
  -e MONGODB_URI="$MONGODB_URI"
  docai-backend
```

API Reference

`POST /upload/`

Upload a file for extraction.

Form-Data: `file` (binary)

Response (example):

```
{
  "job_id": "2f7c...",
  "filename": "invoice.pdf",
  "document_type": "invoice",
  "ocr_source": "tesseract",
  "fallback_triggered": false,
  "donut_used": true,
  "verification_score": 0.83,
  "final_structured_data": { /* merged JSON */ }
}
```

`POST /feedback/`

Submit field-level feedback from the dashboard.

Body (example):

```
{
  "job_id": "2f7c...",
  "field_path": "/line_items/0/amount",
  "correct_value": "145.00",
  "note": "OCR misread 'S' as '5'"
}
```

Response: `{ "status": "saved" }`

`GET /dashboard`

Renders the in-app dashboard (HTML) to review recent extractions and submit feedback.

Data Model (MongoDB)

Database: `docai`

Collections - `ocr_results` — one document per upload/job - `job_id` (string) - `filename` (string) - `timestamp` (ISODate) - `ocr_source` (string) - `document_type` (string) - `verification_score` (float) - `final_structured_data` (object) - additional extraction metadata (engine/layout/pages, etc.)

- `extraction_feedback` — user-submitted corrections
- `job_id` (string)
- `field_path` (JSON pointer-like path, e.g. `/invoice_number` or `/line_items/0/amount`)
- `correct_value` (any)
- `note` (string, optional)
- `timestamp` (ISODate)

  See `Backend/export_feedback.py` for how feedback is merged into verified pairs and exported as JSONL.

Dashboard & Human-in-the-Loop

- `templates/dashboard.html` renders recent jobs with `final_structured_data`.
- Each field exposes controls to submit corrections to `/feedback/`.
- Corrections are stored in `extraction_feedback`.

Verifier (`verifier.py`): - Flattens extracted JSON into string values. - Computes match ratio vs. source OCR text and returns a `verification_score`. - Use this score to flag low-confidence jobs for review.

Training Data Export (JSONL)

Use `Backend/export_feedback.py` to build high-quality training pairs for fine-tuning or RAG exemplars.

It performs roughly: 1. Load original `final_structured_data` for a `job_id` from `ocr_results`. 2. Apply each `extraction_feedback` correction by `field_path` to create a corrected version. 3. Emit JSONL entries:

```
{"messages":[
  {"role":"system","content":"You are an intelligent document extractor."},
  {"role":"user","content":"<original JSON>"},
  {"role":"assistant","content":"<corrected JSON>"}
]}
```

4. Write to a file like `verified_feedback_<id>.jsonl`.

Run:

```
cd Backend
python export_feedback.py
```

Logging & Debugging

- Add structured logs around each stage (OCR → classify → extract → merge → verify → persist).
- Use request-scoped `job_id` to correlate logs and DB records.
- For quick checks:
- `python test_mongo.py` (verifies `MONGODB_URI`)
- `python test_openai.py` (verifies API access) — **remove hard-coded keys first**

Testing

- **Unit tests** (suggested):
- `merge_engine.deep_merge` and `merge_outputs` with diverse nested data.
- `verifier.verify_extraction_alignment` match-ratio edge cases.
- `strategy_router` layout detection heuristics.
- **Integration tests**:
- Upload sample PDFs → assert `final_structured_data` shape and non-zero `verification_score`.

  Consider `pytest` + lightweight fixtures (small PDFs) and GitHub Actions for CI.

Security & Secrets

- Never commit `.env` or credentials. Add this to `.gitignore`:

  ```
  .env
  **/.env
  ```

- Rotate any exposed keys (OpenAI, MongoDB) now.
- In Docker, pass secrets at runtime (`-e OPENAI_API_KEY=...`) or use a secrets manager (AWS/GCP/Azure, or Docker/K8s secrets).
- Validate and sanitize file uploads (size/type limits, virus scanning if needed).

Performance Notes

- **OCR**: `pdf2image` + Tesseract/TrOCR can be CPU-heavy. Consider caching page images and using GPU for TrOCR.
- **Donut**: Model weights are large; lazy-load and reuse across requests.
- **Batching**: Multi-page PDFs benefit from page batching for Donut; merge page-level JSONs with `merge_engine`.

- **Token limits**: Truncate classification/extraction prompts (`document_classifier.py` already truncates to ~3k chars).

Troubleshooting

- **Tesseract not found (local)**: Install Tesseract and ensure it's on PATH. On macOS: `brew install tesseract`. On Ubuntu: `sudo apt-get install tesseract-ocr`.
- `pdf2image` **needs poppler**: Install Poppler. macOS: `brew install poppler`. Ubuntu: `sudo apt-get install poppler-utils`.
- `PIL` / `torch` **wheel errors**: Upgrade `pip` and install build tools. Prefer Docker for consistent builds.
- **Mongo connection fails**: Check network/IP allowlist (Atlas) and `MONGODB_URI` formatting.
- **OpenAI 401/429**: Verify key and rate limits; backoff + retries.

Roadmap

- Frontend app (Vuexy/React) for richer review workflows and role-based auth.
- Configurable strategies (YAML) mapping doc types → extraction chains.
- Add PII redaction for stored OCR text.
- Pluggable vector store for grounding + test-time retrieval.
- Async jobs + Celery/RQ for large PDFs.
- Fine-tuning loop with exported JSONL + evaluation harness.

License

Choose a license (e.g., MIT/Apache-2.0) and add `LICENSE` at repo root.

Maintainers

- Owner: *Your Team*
- Contact: _email@